

Guía de Skills de opencode para Creadores de Contenido

Cómo crear, entender y aprovechar los flujos de trabajo especializados de opencode

JA Palazón Ferrando

2026-06-18

Contenidos

1	¿Qué es un skill en opencode?	1
2	Agente vs Skill	2
3	Cuándo usar skill vs agente	2
4	Anatomía de un skill	2
5	Cómo se activa un skill	3
6	Ejemplo práctico	4
7	Ubicación de los skills	4
8	Consejos para escribir descripciones efectivas	5
8.1	Haz esto	5
8.2	Tabla de patrones recomendados	5
9	Resumen rápido	5



Consejo

¿Para quién es esta guía?

Este documento está pensado para documentalistas, escritores técnicos y diseñadoras de documentación. No necesitas saber programar para entenderlo.

1. ¿Qué es un skill en opencode?

Un **skill** (habilidad o destreza) es un conjunto de instrucciones escritas que opencode carga automáticamente cuando detecta que la tarea que le pides encaja con el propósito del skill.

Piénsalo como una **plantilla de conocimiento**:

- Le dice al modelo *cómo* abordar cierto tipo de tareas.
- Le recuerda *qué* convenciones, reglas o pasos seguir.
- Se activa solo, sin que tengas que mencionarlo.

Nota

Los skills no son código ejecutable. Son **texto instructivo** que el modelo lee antes de responder. Es como darle a un asistente un manual de procedimientos antes de que empiece a trabajar.

2. Agente vs Skill

Aspecto	Agente	Skill
¿Qué es?	Un asistente con rol y personalidad definidos	Instrucciones + flujo de trabajo
¿Cómo se activa?	Manualmente, escribiendo <code>@nombre</code>	Automáticamente, según la tarea
¿Tiene herramientas propias?	Sí, puede tener acceso a herramientas y permisos	No, solo aporta contexto textual
¿Puede ejecutar acciones?	Sí, actúa por sí mismo	No, solo guía al modelo principal
Ejemplo	<code>@editor</code> que revisa ortografía	Skill que aplica guía de estilo automáticamente

Aviso

Los skills **no se invocan manualmente**. Si necesitas activar algo explícitamente, probablemente necesitas un **agente**, no un skill.

3. Cuándo usar skill vs agente

Usa un **skill** cuando:

- Tienes conocimiento contextual que debe aplicarse cada vez que aparece cierto tipo de tarea (ej.: “cada vez que generes documentación, sigue estas reglas de estilo”).
- Quieres que el modelo recuerde convenciones sin que tú las tengas que escribir cada vez.
- El flujo de trabajo es predecible y repetitivo.

Usa un **agente** cuando:

- Necesitas que alguien (aunque sea IA) tome un rol activo: revisar, editar, proponer cambios.
- Quieres invocar una capacidad concreta escribiendo `@nombre`.
- El agente necesita herramientas específicas (navegador, shell, etc.).

Consejo

Regla práctica:

skill = *qué saber* (conocimiento pasivo)

agente = *qué hacer* (acción activa)

4. Anatomía de un skill

Cada skill vive dentro de un directorio con su nombre:

```
.opencode/skills/<nombre>/SKILL.md
```

Ejemplo de estructura:

```
.opencode/skills/  
  revisar-estilo/  
    SKILL.md
```

El archivo `SKILL.md` tiene dos partes:

1. **Frontmatter YAML** — metadatos que opencode lee para saber cuándo activar el skill.
2. **Cuerpo en Markdown** — las instrucciones, ejemplos y flujos de trabajo.

```
---  
name: revisar-estilo  
description: >  
  Revisa que la documentación generada siga la guía de estilo interna.  
  Use cuando se genere documentación nueva o se modifique la existente.  
---
```

Skill: Revisión de Estilo

Reglas generales

- Tono: claro, directo, sin jerga innecesaria.
- Todas las cabeceras usan formato "Title Case".
- Los ejemplos de código deben incluir el lenguaje en el bloque.

Flujo de trabajo

1. Lee el documento generado.
2. Identifica infracciones de estilo.
3. Sugiere correcciones específicas.
4. No modifiques el contenido sin permiso explícito.

Nota

El **frontmatter** es el bloque de metadatos entre `---` al inicio del archivo. opencode lo usa para decidir si el skill es relevante para la tarea actual. El **cuerpo** es lo que se inyecta en el contexto del modelo.

5. Cómo se activa un skill

El modelo principal de opencode decide automáticamente si cargar un skill. Lo hace comparando la **descripción** del skill con la tarea que le acabas de pedir.

No puedes forzar la activación de un skill. Depende de:

- Que la descripción del skill coincida con la tarea.
- Que el modelo considere relevante el skill en ese momento.

Aviso

Si necesitas asegurarte de que algo se cargue siempre, considera usar **instrucciones del proyecto** en lugar de un skill. Los skills son contextuales, no obligatorios.

6. Ejemplo práctico

Skill para **documentalistas** que quieren mantener una guía de estilo coherente.

Archivo: `.opencode/skills/estilo-documentacion/SKILL.md`

```
---
name: estilo-documentacion
description: >
  Aplica la guía de estilo de documentación técnica.
  Use cuando generes, edites o revises documentación en español.
  Use ONLY when trabajes con archivos .qmd, .md o .Rmd.
---

# Guía de Estilo para Documentación Técnica

## Voz y tono

- Usa voz activa: "El sistema procesa los datos" (no "los datos son procesados").
- Trata a la persona lectora de "tú".
- Sé concisa: una idea por párrafo.

## Convenciones de formato

- Titulos: `## Title Case` (nunca todo mayúsculas).
- Código: usa bloques con nombre de lenguaje: ` ``python `.
- Énfasis: usa *cursiva* para términos nuevos, **negrita** para acciones importantes.

## Lo que NO debes hacer

- No uses notas al pie en documentación técnica.
- No uses emojis a menos que la usuaria lo solicite.
- No añadas jerga innecesaria.

## Flujo de revisión

1. Identifica el tipo de archivo (` .qmd `, ` .md `, ` .Rmd `).
2. Revisa tono, formato y convenciones según esta guía.
3. Si encuentras una infracción, señala la línea y sugiere una corrección.
4. Pregunta antes de aplicar cambios automáticamente.
```

7. Ubicación de los skills

Hay dos lugares donde puedes colocar skills:

Ubicación	Alcance	Ejemplo de ruta
Proyecto	Solo afecta a ese proyecto	<code>.opencode/skills/<nombre>/SKILL.md</code>
Global	Afecta a todos los proyectos	<code>~/.config/opencode/skills/<nombre>/SKILL.md</code>

- Los skills de **proyecto** van dentro del repositorio o directorio del proyecto. Son ideales para convenciones de equipo.

- Los skills **globales** van en tu directorio personal de configuración. Sirven para reglas que aplicas siempre, como tu guía de estilo personal.

Consejo

Si trabajas con un equipo, usa skills de proyecto y versiona el directorio `.opencode/` con Git. Así todo el mundo comparte las mismas reglas.

8. Consejos para escribir descripciones efectivas

La descripción es lo que opencode usa para decidir si cargar tu skill. Una buena descripción marca la diferencia.

8.1. Haz esto

- **Pon las palabras clave al inicio.**
"Revisa que la documentación siga la guía de estilo..."
"Skill para cuando necesites que alguien revise..."
- **Sé específica.**
"Use cuando generes archivos `.qmd` o `.md` en español."
"Para documentación."
- **Usa frases como "Use cuando..." o "Use ONLY when..."**.
Esto ayuda al modelo a emparejar el skill con la tarea.
- **Menciona el tipo de archivo, lenguaje o formato.**
Ejemplo: "Use ONLY when trabajes con archivos Python (`.py`)."

8.2. Tabla de patrones recomendados

Patrón	Ejemplo
Use cuando...	Use cuando revises documentación existente.
Use ONLY when...	Use ONLY when modifiques archivos <code>.qmd</code> .
Use para...	Use para aplicar convenciones de escritura inclusiva.

Aviso

Una descripción demasiado genérica hará que el skill se active en contextos inapropiados o que no se active nunca. Invertir tiempo en una buena descripción es clave.

9. Resumen rápido

Concepto	Clave
Skill	Instrucciones que se cargan automáticamente según la tarea
Activación	Automática, la decide el modelo
Formato	<code>.opencode/skills/<nombre>/SKILL.md</code> con frontmatter YAML

Concepto	Clave
Ubicación	Proyecto (<code>.opencode/</code>) o global (<code>~/.config/opencode/</code>)
Descripción	Debe ser específica, con keywords al inicio
Diferencia con agente	Skill = conocimiento pasivo; agente = rol activo con herramientas

*Documento generado por el agente **scribe** — opencode*